Introducción

Ricardo Pérez López

IES Doñana, curso 2026/2027

Generado el 2026/11/05 a las 18:11:00

Índice

1. Conceptos básicos **2**
- 1.1. Informática . . . 2
  - 1.1.1. Procesamiento automático . . . 3
- 1.2. Ordenador . . . 4
  - 1.2.1. Definición . . . 4
  - 1.2.2. Funcionamiento básico . . . 4
  - 1.2.3. Representación de información . . . 7
- 1.3. Problema . . . 12
  - 1.3.1. Generalización . . . 12
  - 1.3.2. Ejemplares de un problema . . . 12
  - 1.3.3. Dominio de definición . . . 13
  - 1.3.4. Jerarquías de generalización . . . 13
- 1.4. Algoritmo . . . 14
  - 1.4.1. Definición . . . 14
  - 1.4.2. Características . . . 15
  - 1.4.3. Cualidades deseables . . . 16
  - 1.4.4. Representación . . . 16
  - 1.4.5. Computabilidad . . . 19
  - 1.4.6. Corrección . . . 20
  - 1.4.7. Complejidad . . . 20
- 1.5. Programa . . . 22

2. Paradigmas de programación **23**
- 2.1. Definición . . . 23
- 2.2. Imperativo . . . 25
  - 2.2.1. Estructurado . . . 26
  - 2.2.2. Orientado a objetos . . . 26
- 2.3. Declarativo . . . 26
  - 2.3.1. Funcional . . . 27
  - 2.3.2. Lógico . . . 27
  - 2.3.3. De bases de datos . . . 28

3. Resolución de problemas mediante programación **28**
3.1. Introducción . . . 28
3.2. Especificación . . . 29
3.3. Análisis del problema . . . 30
3.4. Diseño del algoritmo . . . 30
3.5. Verificación . . . 31
3.6. Estudio de la eficiencia . . . 31
3.7. Codificación . . . 31
3.7.1. Implementación . . . 33
3.8. Traducción y ejecución . . . 34
3.9. Pruebas . . . 35
3.10. Depuración . . . 35
3.11. Documentación . . . 36
3.12. Mantenimiento . . . 36
3.13. Ingeniería del software . . . 36

1. Conceptos básicos

1.1. Informática

Definición:

> **Informática**:
>
> La ciencia que estudia los sistemas de procesamiento automático de la información, también llamados **sistemas informáticos**.

Estos sistemas están formados por:

- Elementos físicos (***hardware***, lo que se puede tocar), que incluye:
  - CPU, memoria, almacenamiento, dispositivos de entrada/salida (*hardware* en sentido estricto).
  - Redes y conectividad (***netware***).
- Elementos lógicos (***software***, lo que no se puede tocar), que incluye:
  - Programas (*software* en sentido estricto).
  - Datos e información (***dataware***).
- Elementos humanos (***humanware***, profesionales y usuarios).

> En este módulo, cuando hablemos de *software* en general, nos estaremos refiriendo a **programas**.

También hay otros componentes menos conocidos como:

- Programas implementados en hardware (***firmware***).
- Documentación, protocolos, normas, flujos de trabajo (**orgware**).

Nuestra tarea como programadores es la de escribir programas que resuelvan los problemas de los usuarios.

1.1.1. Procesamiento automático

Los sistemas informáticos (en particular, los ordenadores que los forman) procesan la información de forma automática siguiendo las indicaciones de un programa.

Un programa está formado por **instrucciones** que dictan *qué* operaciones hay que realizar con la información, *cuándo* y *cómo*.

Esa información se codifica, almacena y manipula en forma de **datos**.

Datos:

Información codificada y almacenada en un formato y en un soporte adecuados para ser manipulados por un sistema informático.

Instrucciones y datos son los dos componentes esenciales de todo procesamiento automático.

El procesamiento automático de la información siempre sigue el mismo esquema:



El **objetivo** del procesamiento automático de la información es **convertir los *datos de entrada* en *datos de salida*** mediante un *hardware* que ejecuta las instrucciones definidas por un *software* (**programas**).

Los programas gobiernan el funcionamiento del *hardware*, indicándole qué tiene que hacer y cómo.

La **Programación** es la ciencia y el arte de diseñar dichos programas.

Ejemplos

Calcular la suma de cinco números:

- **Entrada**: los cinco números.
- **Proceso**: sumar cada número con el siguiente hasta acumular el resultado final.
- **Salida**: la suma calculada.

Dada una lista de alumnos con sus calificaciones finales, obtener otra lista ordenada de mayor a menor por la calificación obtenida y que muestre sólo los alumnos aprobados:

- **Entrada**: Una lista de pares *(Nombre del alumno, Calificación)*.
- **Proceso**: Eliminar de la lista los pares que tengan una calificación menor que cinco y ordenar la lista resultante de mayor a menor según la calificación.
- **Salida**: la lista ordenada de alumnos aprobados.

Ejercicio

1. Identificar la entrada, el proceso y la salida en los siguientes supuestos:
   a. Convertir una temperatura en grados Fahrenheit a Celsius.
   b. Calcular el área de un triángulo a partir de su base y su altura.
   c. Calcular el perímetro de un cuadrado.
   d. Determinar si una llamada entrante en un teléfono móvil es sospechosa de *spam*.

1.2. Ordenador

1.2.1. Definición

Ordenador:

Un ordenador es una máquina que procesa información automáticamente de acuerdo con un programa.

En el ordenador, el programa está *almacenado* en la misma memoria interna que los datos que manipula (arquitectura de **Von Neumann**).

1.2.2. Funcionamiento básico

1.2.2.1. Elementos funcionales

Un ordenador consta de tres componentes principales:

1. **Unidad central de proceso (CPU) o procesador**
   - *Unidad aritmético-lógica (ALU)*
   - *Unidad de control (UC)*
2. **Memoria**
   - *Memoria principal* o central
     - *Memoria de acceso aleatorio (RAM)*
     - *Memoria de sólo lectura (ROM)*
   - *Memoria secundaria* o externa
3. **Dispositivos de E/S**
   - *Dispositivos de entrada*
   - *Dispositivos de salida*

1.2.2.2. Unidad central de proceso (CPU) o procesador

Unidad aritmético-lógica (ALU):

Realiza los cálculos y el procesamiento numérico y lógico.

Unidad de control (UC):

Ejecuta de las instrucciones enviando las señales a las distintas unidades funcionales involucradas.

1.2.2.3. Memoria

Memoria principal o central:

- De menor capacidad pero mayor velocidad que la memoria secundaria.
- La CPU puede acceder directamente a la información almacenada en la memoria principal.

Memoria secundaria, externa o masiva:

- Más lenta que la memoria principal, pero de mucha más capacidad.
- La información debe pasar primero de la memoria secundaria a la principal para que la CPU pueda trabajar con ella.

Memoria principal

Está formada por celdas o posiciones numeradas.

Cada celda almacena cierta cantidad de información (la misma cantidad todas las celdas).

El número que identifica cada celda se denomina **dirección** de memoria.

Para acceder a la información almacenada en la memoria, la CPU debe indicar la dirección de la celda a la que desea acceder.

La memoria principal almacena los datos y los programas que los manipulan.

Ambos (datos y programas) deben estar en la memoria principal para que la CPU pueda acceder a ellos.

Dos tipos de memoria principal:

- **Memoria de acceso aleatorio (RAM)**:

  Su contenido se borra al apagar el ordenador.

- **Memoria de sólo lectura (ROM)**:

  Información permanente (ni se borra ni se puede cambiar).

  Contiene la información esencial (datos y software) para que el ordenador pueda arrancar.

1.2.2.4. Dispositivos de E/S

Dispositivos de entrada:

Introducen datos en el ordenador (*ejemplos*: teclado, ratón, escáner...)

Dispositivos de salida:

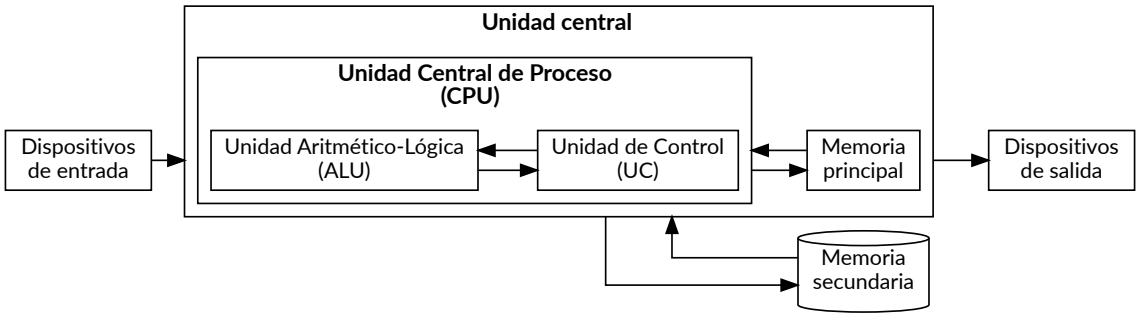
Vuelcan datos fuera del ordenador (*ejemplos*: pantalla, impresora...)

Dispositivos de entrada/salida:

Actúan simultáneamente como dispositivos de entrada y de salida (*ejemplos*: pantalla táctil, adaptador de red...)

Los dispositivos que acceden a **soportes de almacenamiento masivo** (las **memorias secundarias**) también se pueden considerar dispositivos de E/S:

- Los soportes de **sólo lectura** se leen con dispositivos de entrada (*ejemplo*: discos ópticos).
- Los soportes de **lectura/escritura** operan como dispositivos de entrada/salida (*ejemplos*: discos duros, pendrives, tarjetas SD...).



Esquema básico de un ordenador

El programa se **carga** de la memoria secundaria a la memoria principal.

Una vez allí, la CPU va **extrayendo** las instrucciones que forman el programa y las va **ejecutando** paso a paso, en un bucle continuo que se denomina **ciclo de instrucción**.

Durante la ejecución del programa, la CPU recogerá los datos de entrada desde los dispositivos de entrada y los almacenará en la memoria principal, para que las instrucciones puedan operar con ellos.

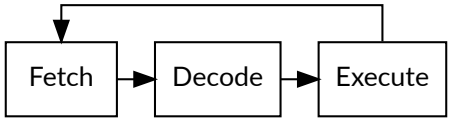
Finalmente, los datos de salida se volcarán hacia los dispositivos de salida.

1.2.2.5. Ciclo de instrucción

En la **arquitectura Von Neumann**, los programas se almacenan en la memoria principal junto con los datos (por eso también se denomina «arquitectura de **programa almacenado**»).

Una vez que el programa está cargado en memoria, la CPU repite siempre los mismos pasos:

1. (**Fetch**) Busca la siguiente instrucción en la memoria principal.
2. (**Decode**) Decodifica la instrucción (identifica qué instrucción es y se prepara para su ejecución).
3. (**Execute**) Ejecuta la instrucción (envía las señales de control necesarias a las distintas unidades funcionales).



Ciclo de instrucción

1.2.3. Representación de información

Los ordenadores son **sistemas electrónicos digitales** que trabajan conmutando entre varios posibles estados de una determinada magnitud física (voltaje, intensidad de corriente, etc.).

A cada uno de esos posibles estados le podemos hacer corresponder (arbitrariamente) un valor numérico.

Por eso, en un sistema informático, toda la información se almacena y se manipula en forma de **números**.

Por ejemplo, cada una de las celdas de la memoria principal puede almacenar un número de un cierto tamaño.

Por tanto, para que un sistema informático pueda procesar información, primero hay que representar dicha información usando números, proceso que se denomina **codificación**.

Codificación:

Proceso mediante el cual se representa información dentro de un sistema informático, asociando a cada dato uno o más valores numéricos según un conjunto de reglas preestablecidas.

Una codificación, por tanto, es una correspondencia entre un conjunto de datos y un conjunto de secuencias numéricas llamado **código**.

Al codificar, lo que hacemos es asociar a cada dato una determinada secuencia numérica dentro del código.

Esa secuencia numérica es **la codificación del dato** en el sistema informático.

Hay muchos tipos de información (textos, sonidos, imágenes, valores numéricos...) y eso hace que pueda haber muchas formas de codificación.

Incluso un mismo tipo de dato (un número entero, por ejemplo) puede tener distintas codificaciones, cada una con sus características y propiedades, y en cada caso puede ser útil usar una u otra.

1.2.3.1. Sistema binario

En un sistema electrónico digital, lo más sencillo y práctico es usar **únicamente dos estados posibles** para la magnitud física que estemos considerando.

Por ejemplo:

- 0 V y 5 V de voltaje.
- 0 mA y 100 mA de intensidad de corriente.

Por tanto, a cada uno de los dos posibles estados le podemos corresponder (arbitrariamente) un valor numérico **0** ó **1**.

A ese valor se le denomina **bit** (contracción de *binary digit*, dígito binario).

Bit:

Un bit es la unidad mínima de información que es capaz de almacenar y procesar un ordenador, y equivale a un **dígito binario**.

Por ejemplo, la memoria principal de un ordenador está formada por millones de celdas.

A su vez, cada celda está formada por un número fijo de componentes electrónicos, con características parecidas a microscópicos condensadores.

Todas las celdas de una misma memoria contienen la misma cantidad de esos condensadores.

Cada uno de estos condensadores puede estar cargado o descargado y, por tanto, es capaz de almacenar un bit:

- Condensador cargado: bit a 1
- Condensador descargado: bit a 0

(La interpretación podría ser al revés, ya que solo es un convenio.)

En consecuencia, cada celda de una memoria puede almacenar una cierta cantidad fija de bits, y todas las celdas de la misma memoria puede almacenar la misma cantidad de bits.

Un bit es una unidad muy pequeña, así que en la práctica se usan **múltiplos del bit**:

- 1 byte = 8 bits
- 1 Kibibyte (KiB) = 2^{10} bytes = 1024 bytes
- 1 Mebibyte (MiB) = 2^{20} bytes = 1024 Kibibytes
- 1 Gibibyte (GiB) = 2^{30} bytes = 1024 Mebibytes
- 1 Tebibyte (TiB) = 2^{40} bytes = 1024 Gibibytes

El sistema de numeración que usamos habitualmente los seres humanos es el **decimal** o sistema **en base diez**.

En ese sistema disponemos de diez dígitos distintos (0, 1, 2, 3, 4, 5, 6, 7, 8 y 9) y cada dígito en un determinado número tiene un peso que es múltiplo de una potencia de diez.

Por ejemplo:

$$243 = 2 \times 10^2 + 4 \times 10^1 + 3 \times 10^0$$

El sistema de numeración que usamos para contar con los dedos es el **unario** o sistema en **base uno** (poco práctico).

El sistema de numeración que usan los ordenadores es el **sistema binario** o sistema **en base dos**, en el cual disponemos sólo de dos dígitos (0 y 1) y cada peso es múltiplo de una potencia de dos.

Por ejemplo:

$$101 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

El mismo dato se puede codificar de distintas formas según convenga.

Generalmente, los **números naturales** se codifican internamente mediante su representación en binario.

Los **números enteros** se suelen codificar mediante:

- Bit de signo (signo y magnitud)
- Complemento a uno
- Complemento a dos

Los **números reales** se pueden codificar mediante:

- Coma fija
- Coma flotante (de simple o doble precisión)

El decimal codificado en binario (BCD) codifica **cada dígito por separado** en binario usando 4 bits por dígito.

La **información textual** (cualquier texto escrito con letras, dígitos y otros símbolos) se representa en forma de **cadenas de caracteres**.

La representación interna de un dato puede ser distinta de:

- La forma en la que se almacena ese dato en soportes secundarios.

  Para esto se utilizan **formatos de almacenamiento**.

- La forma en la que se transmite entre sistemas o entre componentes del mismo sistema.

  Para esto se utilizan **formatos de intercambio**.

A veces, un mismo formato puede ser de almacenamiento y de intercambio al mismo tiempo.

Dependiendo del tipo de información a representar, se pueden usar:

- **Formatos de texto plano** para información estructurada que pueda representarse con cadenas de caracteres: XML, HTML, JSON, CVS, etc.

  Su característica principal es que resulta *legible* directamente para el ser humano.

- **Formatos binarios** estructurados para imágenes (JPEG, PNG), audio (MP3, OPUS), vídeo (OGM, AVI, WebM), etc.

  Al contrario que los formatos de texto plano, los formatos binarios no se pueden leer directamente como si fueran cadenas de caracteres en texto plano, sino que deben ser interpretados mediante programas que lean y entiendan ese formato.

1.2.3.2. Conjuntos de caracteres

Las cadenas de caracteres se usan para representar información textual tanto en el interior de la memoria (codificación interna) como externamente (codificación externa).

Para representar cadenas de caracteres, el ordenador hace corresponder a cada carácter (letra, dígito, signo de puntuación, símbolo especial...) un *código* (que es un número) dentro de un **conjunto de caracteres**.

Esos conjuntos de caracteres pueden tener un esquema de codificación de longitud fija o variable:

- De **longitud fija**: todos los códigos tienen la misma longitud (mismo número de bytes).
- De **longitud variable**: en el mismo conjunto de caracteres hay códigos más largos y más cortos (por tanto, hay caracteres que ocupan más bytes que otros).

Ejemplos de conjuntos de caracteres más utilizados son ASCII, EBCDIC, ISO-8859-1 y Unicode.

ASCII

American Standard Code for Information Interchange.

El conjunto de caracteres ASCII (o **código ASCII**) es el más implantado en el *hardware* de los equipos informáticos.

Es la base de otros códigos más modernos, como el ISO-8859-1 o el Unicode.

Es un código de 7 bits:

- Cada carácter ocupa 7 bits.
- Hay $2^7 = 128$ caracteres posibles.
- Los 32 primeros códigos (del 0 al 31) son no imprimibles (códigos de control).

Se denomina **ASCII extendido** a cualquier conjunto de caracteres de 8 bits que extiende el ASCII con un bit más y en el que los códigos 32 a 126 coinciden con los caracteres imprimibles del ASCII.

El **ISO-8859-1** es un ASCII extendido para contener caracteres latinos.

Tabla de caracteres ASCII estándar de 7 bits:

Dec	Hex	Car	Dec	Hex	Car	Dec	Hex	Car	Dec	Hex	Car
32	20		56	38	8	80	50	P	104	68	h
33	21	!	57	39	9	81	51	Q	105	69	i
34	22	"	58	3A	:	82	52	R	106	6A	j
35	23	#	59	3B	;	83	53	S	107	6B	k
36	24	$	60	3C	<	84	54	T	108	6C	l
37	25	%	61	3D	=	85	55	U	109	6D	m

```
38  26   &     62  3E   >     86  56   V     110 6E   n
39  27   '     63  3F   ?     87  57   W     111 6F   o
40  28   (     64  40   @     88  58   X     112 70   p
41  29   )     65  41   A     89  59   Y     113 71   q
42  2A   *     66  42   B     90  5A   Z     114 72   r
43  2B   +     67  43   C     91  5B   [     115 73   s
44  2C   ,     68  44   D     92  5C   \     116 74   t
45  2D   -     69  45   E     93  5D   ]     117 75   u
46  2E   .     70  46   F     94  5E   ^     118 76   v
47  2F   /     71  47   G     95  5F   _     119 77   w
48  30   0     72  48   H     96  60   `     120 78   x
49  31   1     73  49   I     97  61   a     121 79   y
50  32   2     74  4A   J     98  62   b     122 7A   z
51  33   3     75  4B   K     99  63   c     123 7B   {
52  34   4     76  4C   L     100 64   d     124 7C   |
53  35   5     77  4D   M     101 65   e     125 7D   }
54  36   6     78  4E   N     102 66   f     126 7E   ~
55  37   7     79  4F   O     103 67   g
```

¡CUIDADO!

Es importante distinguir entre (por ejemplo):

1. el número entero 36 y
2. la cadena de caracteres 36, formada por los caracteres 3 y 6.

En el primer caso, el número 36 se codificaría en complemento a dos mediante la secuencia de bits 100100.

En el segundo caso, la cadena 37 se representaría mediante los bytes 00110011 y 00110110 según el código ASCII (que en decimal son 51 y 54, respectivamente).

Es importante darse cuenta de que la misma secuencia de bits se puede interpretar de distintas formas (como un número, como un carácter, etc.) según el contexto o el programa usado.

Unicode

Con 8 bits (y con 7 bits aún menos) no es posible representar todos los posibles caracteres de todos los sistemas de escritura usados en el mundo.

Unicode es el estándar de codificación de caracteres más completo y universal en la actualidad.

Cada carácter en Unicode se define mediante un identificador numérico llamado *code point*.

Unicode define tres esquemas de codificación:

- **UTF-8**: codificación de 8 bits, de longitud variable (cada *code point* puede ocupar de 1 a 4 bytes). **El más usado en la actualidad.**
- **UTF-16**: codificación de 16 bits, de longitud variable (cada *code point* puede ocupar 1 ó 2 palabras de 16 bits).
- **UTF-32**: codificación de 32 bits, de longitud fija (cada *code point* ocupa 1 palabra de 32 bits).

Ejercicios

2. ¿Cómo se podría representar (codificar) lo siguiente?
   a. Un color.
   b. Una imagen.
   c. Un vídeo.
   d. Una nota musical.
   e. Una partitura.
   f. Una grabación de audio.

1.3. Problema

Escribimos programas para que el ordenador procese información de forma automática.

Pero ese procesamiento automático se lleva a cabo por una razón: **resolver un problema** usando un ordenador.

Si un problema es **resoluble** usando un ordenador (*no todos lo son*), podremos escribir un programa que lo resuelva.

Estos son los problemas que nos interesa estudiar en Programación.

1.3.1. Generalización

En Programación nos interesa siempre resolver problemas generales y no casos particulares.

Por ejemplo, el problema de calcular la suma de 4 y 3 es un **problema particular**, porque una solución al problema sólo servirá para resolver *ese* problema en concreto, y no servirá para sumar otro par de números (el 9 y 5, por ejemplo).

En cambio, el problema de calcular la suma de cualquier par de números enteros es un **problema general**, ya que una solución al problema serviría para resolver cualquier caso particular de ese problema general.

Por ejemplo, esa solución al problema general me serviría para calcular la suma de 4 y 3, de 9 y 5, de 12 y 38, … De hecho, infinitos casos particulares.

1.3.2. Ejemplares de un problema

A los casos particulares de un problema general se les denomina **ejemplares** del problema.

Por ejemplo, la pareja $(4, 3)$ es un ejemplar del problema general de sumar dos números enteros.

Normalmente, un problema consistirá en una colección infinita de ejemplares.

Pero también hay problemas finitos (aunque muy grandes) como el de jugar perfectamente al ajedrez.

La solución a un problema debe resolver correctamente todos los ejemplares del mismo, es decir, debe resolver el problema general de forma que sirva para todos sus ejemplares.

1.3.3. Dominio de definición

El **dominio de definición** de un problema describe con precisión el conjunto de sus ejemplares.

Por ejemplo: en el problema de sumar dos números enteros, sus ejemplares serán cualquier pareja de números enteros (no vale que los números sean reales o fracciones). Ese es su dominio de definición.

La solución al problema debe centrarse en el dominio de definición del problema, y no está obligado a resolver ejemplares que se encuentren fuera de dicho dominio de definición.

Por ejemplo: un programa que resuelva correctamente el problema de sumar dos números enteros no tiene por qué funcionar correctamente si intentamos usarlo para sumar dos fracciones.

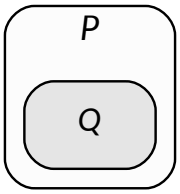
1.3.4. Jerarquías de generalización

Se dice que un problema *P* es **más general** que un problema *Q* (o bien, que es una **generalización** del problema *Q*) si los ejemplares de *Q* también son ejemplares de *P*, y además hay ejemplares de *P* que no lo son de *Q*.

Es otra forma de decir que el conjunto de ejemplares de *Q* es un *subconjunto propio* del conjunto de ejemplares de *P*.

Igualmente, se dice que *Q* es un problema **menos general** o **más particular** que el problema *P* (o bien, que es una **especialización** del problema *P*).

Este concepto resulta interesante porque, si tenemos una solución al problema *P*, podremos usarla para resolver el problema *Q*.

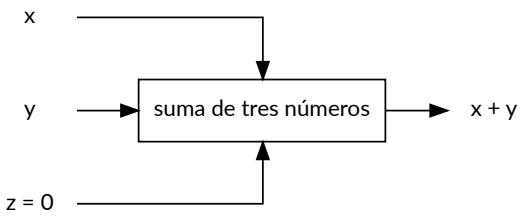


P es un problema más general que *Q*

Por ejemplo, el problema de calcular la suma de *tres* números enteros es un problema **más general** que el de sumar *dos* números enteros, porque éste último se puede considerar un caso particular del primero (haciendo que uno de los tres números a sumar valga cero).

Por tanto, si tenemos un método para resolver el problema más general (el de sumar tres números) podemos usarlo para resolver uno menos general (el de sumar dos números).

En este caso, basta con hacer que uno de los tres números sea cero y los otros dos sean los números a sumar:



Sumar dos números es un caso particular de sumar tres

Ejercicios

3. Con cada uno de los siguientes problemas, dar al menos un ejemplar del mismo e inventar un problema más general:
   a. Calcular cuántos días han pasado entre dos fechas del mismo año.
   b. Calcular el perímetro de un cuadrado a partir de la longitud de uno de sus lados.
4. Inventar un problema más particular para el problema de sumar dos fracciones de números enteros y dar dos ejemplares distintos de cada uno de los dos problemas.
5. Dado el siguiente problema: «Calcular cuántos picos y cuántas patas hay en una granja con *X* gallinas y *Y* cerdos», determinar si los siguientes casos son *ejemplares* del problema o bien son *especializaciones* del problema:
   a. Calcular cuántos picos y cuántas patas hay en una granja con 5 gallinas y 7 cerdos.
   b. Calcular cuántas patas hay en una granja con *Z* cerdos.

1.4. Algoritmo

1.4.1. Definición

Algoritmo:

Un algoritmo es un método para resolver un problema.

Está formado por una secuencia de pasos o **instrucciones** que se deben seguir (o **ejecutar**) para resolver el problema.

La palabra «algoritmo» proviene de **Mohammed Al-Khowârizmi**, matemático persa que vivió durante el siglo IX y reconocido por definir una serie de reglas paso a paso para sumar, restar, multiplicar y dividir números decimales.

Euclides, el gran matemático griego (del siglo IV a. C.) que inventó un método para encontrar el máximo común divisor de dos números, se considera con Al-Khowârizmi el otro gran padre de la Algorítmica (la ciencia que estudia los algoritmos).

El estudio de los algoritmos es importante porque la resolución de un problema exige el diseño de un algoritmo que lo resuelva.

Puede haber muchas formas distintas de resolver el mismo problema, por lo que **pueden existir muchos algoritmos distintos que resuelvan el mismo problema**.

Un algoritmo será mejor que otro si es más claro o más eficiente.

Una vez diseñado el algoritmo, se traduce a un programa informático usando un *lenguaje de programación*.

Finalmente, un ordenador ejecuta dicho programa.



Resolución de un problema

1.4.2. Características

Un algoritmo debe ser:

- **Preciso**: debe expresarse de forma no ambigua. La precisión afecta por igual a dos aspectos:
  - Al *orden* de los pasos que han de llevarse a cabo.
  - Al *contenido* de los pasos, pues en cada uno hay que saber qué hacer exactamente.
- **Determinado**: si se sigue un algoritmo dos veces, se debe obtener el mismo resultado cada vez.
- **Finito**: debe terminar en algún momento, es decir, debe tener un número finito de pasos.

Ejemplos

Sumar, restar, multiplicar, dividir.

Algoritmo de Euclides.

Algoritmos de conteo:

- De uno en uno.
- De dos en dos.
- Conteo «divide y vencerás».

Algoritmos de ordenación:

- Inserción directa.
- Selección directa.
- Burbuja.
- Ordenación rápida.

Algoritmos de búsqueda:

- Búsqueda lineal.
- Búsqueda binaria o dicotómica.

Conteo «divide y vencerás»

1. Ponte de pie y recuerda el número 1.
2. Forma una pareja con otra persona que siga de pie, suma su número y el tuyo, y recuerda la suma.
3. Uno de los dos se sienta (sólo uno).
4. Si sigues de pie, vuelve al paso 2.
5. Se termina cuando sólo queda una persona en pie, y su suma será el conteo final.

1.4.3. Cualidades deseables

Corrección: El algoritmo debe solucionar correctamente el problema.

Claridad: Debe ser legible y comprensible para el ser humano.

Generalidad: Un algoritmo debe resolver problemas generales. Por ejemplo, un algoritmo que sume dos números enteros debe servir para sumar cualquier pareja de números enteros, y no, solamente, para sumar dos números determinados, como pueden ser el 3 y el 5.

Eficiencia: Un algoritmo es mejor cuanto menos recursos (tiempo, espacio...) necesita para resolver el problema. Por eso no debe realizar pasos innecesarios ni recordar más información de la necesaria.

Sencillez: Hay que intentar que la solución sea sencilla, aun a costa de perder un poco de eficiencia; es decir, se tiene que buscar un equilibrio entre la claridad y la eficiencia.

Modularidad: Un algoritmo puede formar parte de la solución a un problema mayor. A su vez, dicho algoritmo puede descomponerse en otros si esto favorece a la claridad del mismo.

> «Make it correct, make it clear, make it concise, make it fast. In that order.»
>
> -- Wes Dyer

1.4.4. Representación

Un algoritmo se puede describir usando el **lenguaje natural**, es decir, cualquier idioma humano.

¿Qué problema tiene esta forma de representación?

Ambigüedad

En ciertos contextos la ambigüedad es asumible, pero **NO** cuando el destinatario es un ordenador.

¿Podemos decir que esta receta de cocina es un algoritmo?

Instrucciones para hacer una tortilla:

1. Coger dos huevos.
2. Encender el fuego.
3. Echar aceite a la sartén.
4. Batir los huevos.
5. Echar los huevos batidos en la sartén.
6. Esperar a que se haga por debajo.
7. Dar la vuelta a la tortilla.

8. Esperar de nuevo.
9. Sacar cuando esté lista.

Fin

Ejemplos de **ambigüedad**:

- «Ayer vi a María con los prismáticos.»
- «Lo vio venir montado a caballo.»
- «Hablé con Juan cuando estaba desayunando.»
- «Tengo más de 20 años.»
- «Fuimos al banco y nos sentamos un rato.»
- «El cerdo está listo para comer.»
- «Vendemos telas para muñecas hechas de lana.»

1.4.4.1. Ordinograma

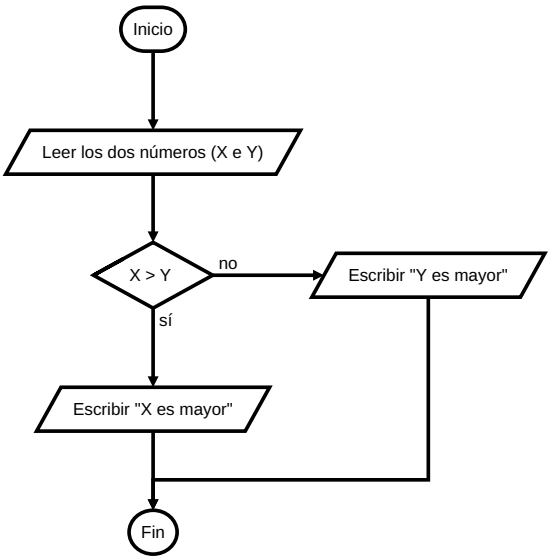
Representación gráfica que describe un algoritmo en forma de diagrama de flujo.

Las flechas indican el orden de ejecución de las instrucciones.

Los nodos condicionales (los rombos) indican que la ejecución se bifurca a uno u otro camino dependiendo de si se cumple o no una condición.

Ejemplo

Determinar cuál es el máximo de dos números



1.4.4.2. Pseudocódigo

Es un **lenguaje *semi-formal***, a medio camino entre el lenguaje natural y el lenguaje que entendería un ordenador (lenguaje de programación).

Está pensado para ser interpretado por una persona y no por un ordenador, así que no hace falta que tenga una sintaxis muy estricta.

En general, no se tienen en cuenta las limitaciones impuestas por el *hardware* (CPU, memoria...) o el *software* (tamaño máximo de los datos, codificación interna...), siempre y cuando no sea importante el estudio de la eficiencia o la complejidad del algoritmo.

En ese sentido, se usa como un lenguaje de programación *idealizado*, es decir, una *abstracción* de un lenguaje de programación real donde no se tienen en cuenta ciertos detalles que resultan innecesarios para entender el funcionamiento del algoritmo.

Por ejemplo, normalmente no nos tendremos que preocupar por el tamaño de los números, así que supondremos que siempre «caben» en la memoria.

Ejemplo

Algoritmo: Obtener el mayor de dos números

1. $X \leftarrow$ **leer** número
2. $Y \leftarrow$ **leer** número
3. **si** $X > Y$ **entonces saltar** al paso 6
4. **escribir** "Y es mayor que X"
5. **saltar** al paso 7
6. **escribir** "X es mayor que Y"
7. **fin**

No existe un único *pseudocódigo*.

A la hora de usar un pseudocódigo para representar un algoritmo, el diseñador del mismo decidirá qué instrucciones son válidas en ese pseudocódigo y qué estilo de programación (lo que luego llamaremos *paradigma*) seguirá el mismo.

Además, es importante es que no haya ninguna duda posible sobre cómo interpretar las instrucciones del pseudocódigo.

Lo más apropiado sería usar un pseudocódigo que se parezca lo más posible al lenguaje de programación con el que finalmente se escribirá el programa, de forma que la tarea de traducir el algoritmo a su correspondiente programa sea lo más fácil y directa posible.

Por ejemplo, el algoritmo anterior sería relativamente fácil de traducir a *lenguaje ensamblador* o *lenguaje máquina*, ya que las instrucciones que se usan en ese pseudocódigo (lecturas, escrituras, saltos...) son fáciles de adaptar a esos lenguajes.

En cambio, sería bastante más complicado traducirlo a un lenguaje funcional como Haskell, donde no existen esas instrucciones.

1.4.5. Computabilidad

¿Todos los problemas pueden resolverse de forma algorítmica?

Dicho de otra forma, queremos saber lo siguiente:

> Dado un problema, ¿existe un algoritmo que lo resuelva?

Todo problema P lleva asociada una función $f_P : D \longrightarrow R$, donde:

- D es el conjunto de los datos de entrada.
- R es el conjunto de los resultados del problema.

Asimismo, todo algoritmo A lleva asociada una función f_A.

La pregunta es: ¿existe un algoritmo A tal que $f_A = f_P$?

Y de ahí vamos a la pregunta general:

> ¿Toda función f es computable (resoluble algorítmicamente)?

La respuesta es que **NO**.

Se puede demostrar que hay más funciones que algoritmos, por lo que **existen funciones que no se pueden computar mediante un algoritmo** (no son computables).

La dificultad que tiene estudiar la computabilidad de funciones está en que no tenemos una definición formal de «*algoritmo*».

A comienzos del S. XX, se crearon (independientemente uno del otro) dos formalismos matemáticos para representar el concepto de *algoritmo*:

- Alonzo Church creó el **cálculo lambda**.
- Alan Turing creó la **máquina de Turing**.

Posteriormente se demostró que los dos formalismos eran totalmente equivalentes y eran, además, equivalentes a las **gramáticas formales**.

Esto llevó a formular la llamada **tesis de Church-Turing**, que dice que

> «*Todo algoritmo es equivalente a una máquina de Turing.*»

La tesis de Church-Turing es indemostrable pero prácticamente toda la comunidad científica la acepta como verdadera.

Usando esos formalismos, se pudo demostrar que hay problemas que no se pueden resolver mediante algoritmos.

Uno de los problemas que no tienen una solución algorítmica es el llamado **problema de la parada**:

> **Problema de la parada**:

Dado un algoritmo y un posible dato de entrada, determinar (a priori, sin ejecutarlo previamente) si el algoritmo se detendrá y producirá un valor de salida.

Nunca podremos hacer un algoritmo que resuelva el problema de la parada en términos generales (en casos particulares sí se puede).

1.4.6. Corrección

¿Cómo sabemos si un algoritmo es **correcto**?

¿Qué significa eso de que un algoritmo sea correcto?

Supongamos que, para un problema P, existe un algoritmo A. Lo que tenemos que averiguar es si se cumple:

$$f_P = f_A$$

¿Cómo lo hacemos?

- Si el conjunto D de datos de entrada es **finito**, podríamos comparar todos los resultados de salida con los resultados esperados y ver si coinciden (**pruebas exhaustivas**). Normalmente es imposible.
- Si D es **infinito**, es imposible realizar una comprobación empírica de la corrección del algoritmo (se pueden realizar **pruebas no exhaustivas** que comprueban algunos datos de entrada pero no todos, por lo que no demuestran que el algoritmo es correcto pero sí pueden demostrar que es incorrecto).

Lo mejor (pero más difícil) es recurrir a **métodos formales**:

- **Diseño a priori**: se construye el algoritmo en base a una demostración (lo que se denomina también **demostración constructiva**).
- **Diseño a posteriori**: se construye el algoritmo de forma más o menos intuitiva y, una vez diseñado, tratar de demostrar su corrección.

En ambos casos, es importante definir con mucha precisión qué problema queremos resolver.

Para ello, se describe el problema mediante una **especificación formal**.

1.4.7. Complejidad

¿Cómo de **eficiente** es un algoritmo?

La eficiencia de un algoritmo se mide en función del **consumo de recursos** que necesita el algoritmo para su ejecución.

Los principales recursos son el **tiempo** y el **espacio**.

Dados dos algoritmos distintos que resuelvan el mismo problema, en general nos interesará usar el más eficiente de ellos (al margen de otras consideraciones, como la claridad, la legibilidad, la mantenibilidad, la reusabilidad, etc.)

¿Cómo medimos la eficiencia de un algoritmo?

¿Cómo comparamos la eficiencia de dos algoritmos?

El **análisis de algoritmos** estudia la eficiencia de un algoritmo desde un punto de vista abstracto (independiente de la máquina, el lenguaje de programación, la carga de trabajo, etc.).

Define el consumo de recursos **en función del tamaño del ejemplar del problema a resolver**.

Por ejemplo:

- Supongamos el problema de comprobar cuántas vocales hay en una frase.
- La **entrada** al algoritmo será la frase (una cadena de caracteres).
- La **salida** será el número de vocales que hay en la cadena.
- Cada una de las posibles cadenas de entrada representan un **ejemplar** del problema a resolver.
- Cabe esperar que el algoritmo tarde más en dar el resultado cuanto más larga sea la cadena de entrada.
- Por tanto, el **tamaño del ejemplar** será la longitud de la cadena.

Si tenemos dos algoritmos A y B (que resuelven el mismo problema anterior) con tiempos de ejecución

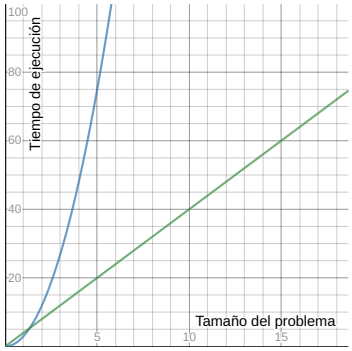
$$t_A(n) \simeq 3n^2$$

y

$$t_B(n) \simeq 4n$$

siendo n la longitud de la cadena de entrada, el algoritmo B se considera más eficiente que A ya que su tiempo de ejecución es menor a medida que aumenta n.

Esto es así aunque hay algún caso (como el de $n = 1$) donde el algoritmo A es más eficiente que B, ya que la constante multiplicativa 3 que aparece en $t_A(n)$ es más pequeña que la constante 4 que aparece en $t_B(n)$.



Representación gráfica de $3n^2$ y $4n$

En general, no estamos interesados en las constantes concretas que puedan aparecer en las $t(n)$, ni en el valor que éstas puedan tomar para un n concreto, sino que tan sólo nos interesa la **forma** que puedan tener las funciones $t(n)$ y cómo crecen a medida que aumenta n.

Por eso, clasificamos el consumo de recursos usando una **notación asintótica**, con la cual podemos ordenar las funciones $t(n)$ según determinados *órdenes de crecimiento* cuando n crece hasta el infinito.

En el ejemplo anterior, tenemos

$$t_A(n) \in O(n^2)$$

que se lee «$t_A(n)$ es del orden de n^2», y

$$t_B(n) \in O(n)$$

que se lee «$t_B(n)$ es del orden de n». Podemos decir también que A tiene **tiempo cuadrático** y B **tiempo lineal**.

Como $O(n) \subset O(n^2)$ (porque n^2 crece más deprisa que n), podemos concluir que $t_B(n) < t_A(n)$ para un valor de n suficientemente grande (o sea, *asintóticamente*). Por tanto, B es un algoritmo más eficiente que A.

1.5. Programa

Un **programa** es una secuencia de instrucciones que le dicen a un ordenador lo que tiene que hacer para realizar una tarea o resolver un problema.

Esas instrucciones deben estar escritas en un lenguaje que sea entendible por un ordenador, o bien que se pueda traducir de forma automática en algo que pueda entender el ordenador.

Esos lenguajes son los llamados **lenguajes de programación**.

Los algoritmos están pensados para ser entendidos por un ser humano, y se representan mediante técnicas (como el *pseudocódigo*) que están pensadas para ser entendidas por un humano, y no por un ordenador.

En cambio, los programas están escritos para que los entienda un ordenador, para que los interprete y los ejecute.

Sabiendo eso, una definición más precisa de *programa* sería:

> **Programa**:
>
> Un programa es la codificación de un algoritmo en un lenguaje de programación.

El texto del programa escrito en ese lenguaje de programación se denomina **programa fuente** o **código fuente**.

Los *programadores* producen código fuente como resultado de su trabajo, aunque en realidad «programar» es una tarea mucho más compleja que consta de varios pasos, no sólo teclear código fuente.

Al proceso de escribir el código fuente de un programa se le denomina ***codificación***, pero no hay que confundirlo con la *codificación de información*.

Aquí, «*codificar*» consiste en *reescribir* un algoritmo en un programa escrito en un determinado lenguaje de programación.

Por ello, a veces se usa el término *codificar* como sinónimo de *programar*, si bien no son exactamente lo mismo.

Si el algoritmo está bien definido, la codificación puede resultar trivial.

Por ello, toda posible ambigüedad que pudiera quedar en el algoritmo debe eliminarse al codificarlo en forma de programa.

Programar depende mucho de las características del lenguaje de programación elegido.

Lo ideal es usar un lenguaje que se parezca lo más posible al *pseudocódigo* utilizado para describir el correspondiente algoritmo.

En un programa también hay que considerar **aspectos y limitaciones** que no se suelen tener en cuenta en un algoritmo:

- **El tamaño de los datos en memoria:** por ejemplo, en un lenguaje de programación suele haber límites en cuanto a la cantidad de dígitos que puede tener un número o su precisión decimal.
- **Restricciones en los datos:** dependiendo del tipo de los datos, pueden ser mutables o inmutables, de tamaño fijo o variable, etc.
- **La semántica de las instrucciones:** un símbolo usado en un algoritmo puede tener otro significado distinto en el programa, o puede que sólo pueda usarse en el programa bajo ciertas condiciones que no hace falta considerar en el algoritmo.

2. Paradigmas de programación

2.1. Definición

Paradigma de programación:

Es un ***estilo*** de desarrollar programas, es decir, un **modelo** para resolver problemas computacionales.

Cada paradigma entiende la programación desde una perspectiva diferente, partiendo de unos conceptos básicos diferentes y con unas reglas diferentes.

Cuando diseñamos un algoritmo o escribimos un programa, lo hacemos siguiendo un determinado paradigma, y éste impregna por completo la forma en la que describimos la solución al problema en el que estamos trabajando.

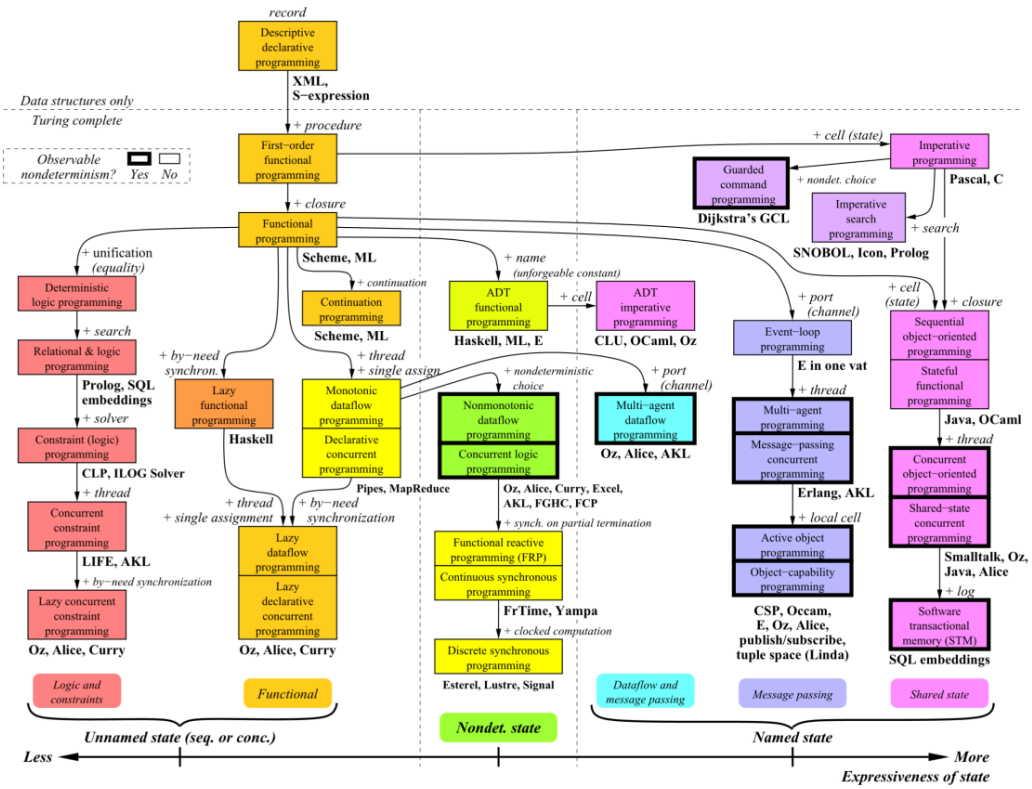
No existe un único paradigma de programación y cada uno tiene sus peculiaridades que lo hacen diferente.

Un **lenguaje de programación (o pseudocódigo)** se dice que ***soporta* un determinado paradigma** cuando con dicho lenguaje se pueden escribir algoritmos o programas según el «estilo» que impone dicho paradigma.

Incluso existen **lenguajes *multiparadigma*** capaces de soportar varios paradigmas al mismo tiempo.

Los paradigmas de programación más importantes son:

- Paradigmas
  - Imperativo
    - Estructurado
    - Procedimental
    - Orientado a objetos
  - Declarativo
    - Funcional
    - Lógico
    - De bases de datos



Principales paradigmas © 2008 Peter Van Roy

Las diferencias fundamentales entre los paradigmas declarativos e imperativos son:

- **Paradigmas declarativos:**
  - Se centran en las **propiedades y relaciones** que se establecen entre los elementos que forman el problema y la solución.
  - Escribir un programa declarativo consiste en describir **qué** forma tiene la solución.
- **Paradigmas imperativos:**

- Se centran en los **pasos detallados** (secuencia de instrucciones) que hay que realizar para obtener la solución.
- Escribir un programa imperativo consiste en describir **cómo** se debe obtener la solución.

Ejemplo

Supongamos que vamos a un restaurante porque queremos una tarta de chocolate.

El enfoque declarativo sería decirle al cocinero: «*quiero una tarta de tres chocolates con virutas por encima*».

En cambio, el enfoque imperativo sería darle al cocinero una lista de ingredientes y una receta detallada para que la siga al pie de la letra y pueda cocinar la tarta de chocolate que yo quiero.

Ejemplo

La raíz cuadrada de un número x, representada como $\sqrt{x}$, es el valor y que cumple:

$$y^2 = x, \quad y \geq 0$$

Esta es la definición de raíz cuadrada: un **conocimiento declarativo** que, como tal, **expresa que algo es «verdad»**.

Pero esta definición no nos sirve de mucha ayuda para calcular cuánto vale la raíz cuadrada de un número dado.

En cambio, podemos definir un método para calcular la raíz cuadrada de un número que sería un **conocimiento imperativo**, es decir, no describe «qué es» la raíz cuadrada, sino que describe **«cómo» obtenerla** de forma aproximada:

1. Empezar con una aproximación g.
2. Mejorar la aproximación calculando $\dfrac{g + x/g}{2}$.
3. Seguir mejorando la aproximación hasta que sea lo bastante buena.

2.2. Imperativo

El **paradigma imperativo** está basado en el concepto de **sentencia**. Un programa imperativo está formado por una sucesión de sentencias que se ejecutan en orden y que llevan a cabo una de estas acciones:

- **Cambiar el estado** interno del programa, usualmente mediante la sentencia de *asignación*.
- Cambiar el **flujo de control** del programa, haciendo que la ejecución se bifurque (*salte*) a otra parte del mismo.

La ejecución de un programa imperativo, por tanto, consiste en una sucesión de cambios de estado controlados por mecanismos de control y que dependen del orden en el que se realizan.

Existen varios paradigmas con las características del paradigma imperativo, por lo que podemos decir que **existen varios paradigmas imperativos**.

2.2.1. Estructurado

El **paradigma estructurado** es un paradigma imperativo en el que el flujo de control del programa se define mediante las denominadas **estructuras de control**.

Se apoya a nivel teórico en los resultados del conocido **teorema de Böhm y Jacopini**, que establece que cualquier programa útil se puede escribir usando solamente tres estructuras básicas:

- Secuencia
- Selección
- Iteración

Con estas tres estructuras conseguimos que los programas se puedan leer de arriba abajo como compuestos por **bloques anidados o independientes** que se leen como un todo conjunto.

Su aparición llevó asociada la aparición de una **metodología de desarrollo** según la cual los programas se escriben por niveles de abstracción mediante refinamientos sucesivos y usando en cada nivel sólo las tres estructuras básicas.

2.2.2. Orientado a objetos

El **paradigma orientado a objetos** se apoya en los conceptos de **objeto** y **mensaje**.

Un programa orientado a objetos está formado por una colección de objetos que se intercambian mensajes entre sí.

Los objetos son entidades que existen dentro del programa y que poseen un cierto **estado interno**.

Cuando un objeto envía un mensaje a otro, el objeto que recibe el mensaje reaccionará llevando a cabo alguna acción, que probablemente provocará un **cambio en su estado interno** y que, posiblemente, provocará también el envío de mensajes a otros objetos.

La programación orientada a objetos está vista como una forma natural de entender la programación y es, con diferencia, **el paradigma más usado en la actualidad**, para bien o para mal.

2.3. Declarativo

La **programación declarativa** engloba a una familia de paradigmas de programación de muy alto nivel.

En programación declarativa se describe la solución a un problema definiendo las **propiedades** que debe cumplir dicha solución en lugar de definir las **instrucciones** que se deben ejecutar para resolver el problema.

Se dice que un programa imperativo describe **cómo** resolver el problema, mientras que un programa declarativo describe **qué** forma debe tener la solución al problema.

Para dar forma a la solución, se utilizan formalismos abstractos matemáticos y lógicos, lo que da lugar a los dos grandes paradigmas declarativos: la **programación funcional** y **la programación lógica**.

2.3.1. Funcional

La **programación funcional** es un paradigma de programación declarativa basado en el uso de **definiciones**, **expresiones** y **funciones matemáticas**.

Tiene su origen teórico en el **cálculo lambda**, un sistema matemático creado en 1930 por Alonzo Church.

Los lenguajes funcionales se pueden considerar *azúcar sintáctico* (es decir, una forma equivalente pero sintácticamente más sencilla) del cálculo lambda.

En programación funcional, una función define un cálculo a realizar a partir de unos datos de entrada, con la propiedad de que el resultado de la función sólo puede depender de esos datos de entrada.

Eso significa que una función no puede tener estado interno ni su resultado puede depender del estado del programa.

Además, una función no puede producir ningún efecto observable fuera de ella (los llamados **efectos laterales**), salvo calcular y devolver su resultado.

Esto quiere decir que en programación funcional no existen los efectos laterales, o se dan de forma muy localizada en partes muy concretas e imprescindibles del programa.

Por todo lo expuesto anteriormente, se dice que las funciones en programación funcional son **funciones puras**.

En consecuencia, es posible sustituir cualquier expresión por su valor, propiedad que se denomina **transparencia referencial**.

La programación funcional es un paradigma cada vez más utilizado, y hasta los lenguajes que no son funcionales están incorporando características propias de este paradigma.

Esto se debe a que demostrar la corrección de un programa funcional o paralelizar su ejecución es **mucho más fácil** que con un programa imperativo.

2.3.2. Lógico

La **programación lógica** es un paradigma de programación declarativa que usa la **lógica matemática** como lenguaje de programación.

Básicamente, un programa lógico es una colección de definiciones que forman un conjunto de **axiomas** en un sistema de **deducción lógica**.

Ejecutar un programa lógico equivale a poner en marcha un mecanismo deductivo que trata de **demostrar un teorema** a partir de los axiomas.

Se usa principalmente en **inteligencia artificial**, en **demostración automática** y en **procesamiento del lenguaje natural**.

El más conocido de los lenguajes de programación lógica es **Prolog**.

2.3.3. De bases de datos

Los sistemas de gestión de bases de datos relacionales (SGBDR) disponen de un lenguaje que permite al usuario consultar y manipular la información almacenada.

A esos lenguajes se los denomina **lenguajes de bases de datos** o **lenguajes de consulta**.

El lenguaje de consulta más conocido es el **SQL**.

Los SGBDR se basan en el *modelo relacional*, que es un modelo matemático.

SQL es, básicamente, una implementación del **álgebra relacional**.

Los lenguajes de consulta se consideran lenguajes declarativos porque con ellos el usuario indica *qué* desea obtener (qué propiedades debe cumplir la solución) y el SGBDR determina automáticamente el mejor camino para alcanzar dicho objetivo.

Ejercicios

6. ¿Qué paradigmas de programación soportan los lenguajes Python y Java?
7. ¿Hay más paradigmas de programación? Busca en Internet un par de ejemplos de paradigmas que no se hayan nombrado aquí. Para cada uno, describe en una sola frase sus características básicas.

3. Resolución de problemas mediante programación

3.1. Introducción

El proceso de resolución de un problema con un ordenador pasa por escribir y ejecutar un programa.

Aunque diseñar programas es, esencialmente, un proceso creativo, se puede considerar una serie de fases o pasos comunes, que generalmente deben seguir todos los programadores.

Si bien se han ordenado según un esquema lógico, hay que considerar que algunos de esos pasos se repiten a lo largo del desarrollo del programa siguiendo un esquema iterativo e incremental, y otros (como la documentación) se deben realizar continuamente a lo largo de todo el proceso y no sólo al final como un paso más del mismo.

Los pasos para la resolución de un problema mediante programación son los siguientes:

1. Especificación.
2. Análisis del problema.
3. Diseño del algoritmo.
4. Verificación.
5. Estudio de la eficiencia.
6. Codificación.
7. Traducción y ejecución.
8. Pruebas.
9. Depuración.

10. Documentación.
11. Mantenimiento.

3.2. Especificación

La **especificación de un *problema*** describe **qué** problema hay que resolver sin entrar a detallar **cómo** hay que resolverlo.

La **especificación de un *programa*** describe **qué** tiene que hacer el programa para resolver un problema sin detallar aún **cómo** va a resolverlo.

En esta fase, se ve al programa como una **caja negra** de la que se sabe *qué* debe hacer pero aún no sabemos *cómo* va a hacerlo.

La especificación define con precisión (cuanto más formal mejor):

- Cuál es la **entrada requerida**:
  - Qué datos de entrada se necesitan y qué propiedades deben cumplir.
  - Con esto se determina el **dominio de definición** del problema, es decir, el conjunto de sus ejemplares.
- Cuál es la **salida deseada**:
  - Básicamente, es el resultado que se desea obtener como solución al problema.
  - Normalmente se describe en función de los datos de entrada.

Ejemplo

Se desea determinar el máximo de dos números enteros.

- *Entrada*: los dos números enteros (llamémosles n_1 y n_2).
- *Salida*: el mayor de los dos números.

Se representaría esquemáticamente así:

$$\begin{cases} \textbf{Entrada} : n_1, n_2 \in \mathbb{Z} \\ \textit{Máximo} \\ \textbf{Salida} : \text{el mayor de ambos} \end{cases}$$

En las especificaciones está permitido usar operaciones (funciones, por ejemplo) siempre y cuando estén perfectamente especificadas, aunque no estén implementadas en el lenguaje de programación.

A esas operaciones se las denomina **operaciones auxiliares**.

Por ejemplo, si disponemos de la función auxiliar *max* que devuelve el máximo de dos números, nuestra especificación podría quedar así:

$$\begin{cases} \textbf{Entrada} : n_1, n_2 \in \mathbb{Z} \\ \textit{Máximo} \\ \textbf{Salida} : max(n_1, n_2) \end{cases}$$

Con esta especificación estamos describiendo que, si se reciben como datos de entrada dos números enteros cualesquiera, el programa *Máximo* calculará y devolverá a la salida el mayor de ellos.

Si los datos de entrada no satisfacen los requisitos necesarios, el programa tiene derecho a responder de cualquier manera, o a no responder en absoluto.

3.3. Análisis del problema

A partir de la especificación, se estudia detalladamente el problema a resolver, los requisitos que se deben cumplir y las posibles restricciones que pueda tener la solución.

En el ejemplo anterior:

- Hay que comparar el valor de los dos números y devolver el mayor de ellos.
- Si los dos números son iguales, se puede devolver cualquiera de los dos.

3.4. Diseño del algoritmo

Una vez analizado el problema con detalle, se diseña un algoritmo que lo resuelva, cumpliendo con todas las posibles restricciones y satisfaciendo la especificación del problema.

El algoritmo se representa con cualquier herramienta adecuada para ello (ordinogramas, pseudocódigo, etc.) la cual depende del paradigma utilizado.

El paradigma usado para describir el algoritmo debería ir acorde con el paradigma del lenguaje de programación que se usará luego para codificar el algoritmo en forma de programa.

Por ejemplo, se podría usar un ordinograma para representar un algoritmo imperativo pero no es apropiado para uno funcional.

Igualmente, hay distintos pseudocódigos y cada uno sigue un determinado paradigma y posee un determinado juego de instrucciones válidas, por lo que ciertos pseudocódigos serán más apropiados que otros para ser traducidos luego a un determinado lenguaje de programación.

Por ejemplo, un algoritmo descrito en pseudocódigo siguiendo un estilo estructurado podría ser:

Algoritmo: Cálculo del máximo de dos números
Entrada: $n_1, n_2 \in \mathbb{Z}$
Salida: el mayor de ambos

```
inicio
    si n1 > n2 entonces
        devolver n1
    sino
        devolver n2
fin
```

Por tanto, este algoritmo, así representado, sería muy apropiado para traducirlo luego a un programa escrito en un lenguaje estructurado (como Python o Java), pero no tanto para traducirlo a un lenguaje funcional como Haskell.

3.5. Verificación

Es el proceso por el cual se intenta **demostrar** que el algoritmo diseñado es **correcto**.

Un algoritmo es correcto cuando **satisface su especificación**.

Es un proceso basado en las matemáticas y la lógica, y consiste en considerar que el algoritmo es un **teorema** a demostrar en un sistema deductivo lógico en el que hay *axiomas* y *reglas de inferencia*.

Puede resultar muy difícil incluso en casos sencillos.

En la práctica, su uso se reduce a bloques pequeños y críticos del programa.

3.6. Estudio de la eficiencia

Cuando disponemos de un algoritmo correcto que resuelve el problema, podemos optar por estudiar la eficiencia del mismo (es decir, la cantidad de recursos que consume).

Si el algoritmo es correcto pero ineficiente, no suele resultar práctico, y se debe optar por diseñar otro algoritmo más eficiente.

Los algoritmos ineficientes sólo resultan útiles cuando el tamaño del ejemplar del problema es relativamente pequeño.

Hay que tener en cuenta que existen problemas para los que no se conoce ningún algoritmo eficiente.

También puede resultar muy interesante estudiar la eficiencia cuando tenemos varios algoritmos que resuelven el mismo problema y queremos determinar cuál de ellos es *mejor*.

En ese sentido, la eficiencia sería un criterio (entre otros) a la hora de comparar algoritmos para determinar si un algoritmo es mejor que otro.

Otros criterios importantes son la claridad, la elegancia o la reusabilidad.

Es importante recordar que hay una regla no escrita que dice que un algoritmo más eficiente suele ser menos claro o elegante, y viceversa.

3.7. Codificación

Una vez diseñado y verificado, **el algoritmo se codifica en un lenguaje de programación** usando un editor de textos o un IDE (*Entorno Integrado de Desarrollo*).

Se considera una tarea casi mecanizable, pero aún hay decisiones que pueden influir a la hora de codificar un programa y que sólo puede tomar un programador experimentado.

El lenguaje de programación utilizado es una decisión de diseño que hay que justificar.

En teoría, el diseño del algoritmo debería ser independiente del lenguaje de programación en el que se vaya a codificar posteriormente el programa, pero el *estilo* (paradigma) utilizado influye mucho.

Por tanto, en la práctica se procura que el algoritmo esté escrito en el mismo paradigma que sigue el lenguaje de programación que se va a usar para codificar el programa.

Por otra parte, el lenguaje de programación o la arquitectura hardware donde se va a ejecutar el programa pueden incorporar **restricciones o condiciones** que hasta ahora no se habían tenido en cuenta al diseñar el algoritmo.

Por ejemplo, es muy común que los lenguajes de programación impongan un tamaño máximo de almacenamiento de los datos que maneja.

En el caso del problema de sumar dos números enteros, eso significa que el programa podría no admitir números demasiado grandes, lo que habría que tenerlo en cuenta a la hora de escribir el programa.

Ese detalle no lo consideramos al diseñar el algoritmo, ya que **los algoritmos son *ideales* y se representan con herramientas *ideales*** (los pseudocódigos).

Un programador experimentado podría saltarse el paso de representar el algoritmo usando pseudocódigo y podría pasar directamente a escribir el correspondiente programa en un lenguaje de programación.

En el pasado, los lenguajes de programación eran poco expresivos y poco ricos en tipos de instrucciones y estructuras de datos, lo que hacía que traducir un algoritmo escrito en pseudocódigo a un programa resultara una trabajo más costoso y menos directo.

Asimismo, el programa resultante era mucho menos claro y difícil de entender que su algoritmo equivalente escrito en pseudocódigo.

Hoy día existen lenguajes de programación de alto nivel muy expresivos y con una sintaxis muy clara y legible que funcionan casi como «pseudocódigos directamente interpretables por el ordenador».

Python, por ejemplo, se considera un ejemplo de este tipo de lenguajes.

Codificación del algoritmo anterior en lenguaje Python:

```python
def maximo(n1, n2):
    """Calcula el máximo de dos números enteros."""
    if n1 > n2:
        return n1
    else:
        return n2
```

Codificación en lenguaje Java:

```java
/*
 * Calcula el máximo de dos números enteros.
 */
public static int maximo(int n1, int n2) {
    if (n1 > n2) {
        return n1;
    } else {
        return n2;
    }
}
```

Codificación en lenguaje Haskell:

```haskell
-- Calcula el máximo de dos números enteros.
max :: (Ord a) => a -> a -> a
max a b
    | a > b     = a
    | otherwise = b
```

Codificación en lenguaje Scheme:

```scheme
; Calcula el máximo de dos números enteros.
(define (maximo n1 n2)
  (cond ((> n1 n2) n1)
        (else n2)))
```

3.7.1. Implementación

El concepto de **implementación** es muy importante en Programación.

Está muy relacionado con los conceptos de *especificación*, *diseño* y *codificación*.

En esencia:

- La *especificación* describe *qué* hay que hacer.
- La *implementación*, en cambio, describe *cómo* hay que hacerlo.

Cuando lo que se está especificando es un *programa* (es decir, cuando la especificación describe lo que tiene que hacer un programa), la *implementación* es el *algoritmo* o *programa* que hace lo que la especificación dice que hay que hacer.

En ese sentido, *implementar* puede verse como sinónimo de *realizar*, *diseñar* o *codificar*.

Por tanto, *programar* consiste en: primero, *especificar* y después *implementar*.

En ese sentido, la implementación va asociada siempre a una especificación, ya que debe satisfacer a ésta.

Decimos que **una implementación *satisface* a una especificación** cuando dicha implementación hace lo que exige esa especificación, es decir, cuando la implementación proporciona la salida deseada a partir de los datos de entrada requeridos.

Recordemos que los datos de entrada y la salida deseada vienen descritos en la especificación.

En Programación podemos implementar algoritmos o programas (y también otras cosas que ya veremos posteriormente).

«*Implementar un algoritmo*» e «*implementar un programa*» son expresiones que a veces se suelen usar indistintamente, ya que tienen un significado similar o casi idéntico.

Por ejemplo, podemos decir que «implementamos un algoritmo» cuando diseñamos un algoritmo a partir de una especificación y lo representamos usando una herramienta apropiada (pseudocódigo, diagramas, etc.). En este caso, se está usando «implementar» como sinónimo de «diseñar un algoritmo».

Pero también podemos decir que «implementamos un algoritmo» cuando ya tenemos un algoritmo y lo codificamos en un determinado lenguaje de programación, creando así un programa. En este caso, se está usando «implementar» como sinónimo de «codificar».

Y también podemos decir que «implementamos un programa» cuando el producto resultante es un programa, ya sea a partir de una especificación o de un algoritmo ya diseñado a partir de esa especificación.

Por tanto, «implementar» es el acto de crear algo a partir de una especificación. El producto resultante (la implementación) puede ser un algoritmo, un programa o más cosas que ya veremos en su momento.

Puede haber varias implementaciones (algoritmos) que satisfagan la misma especificación.

Asimismo, puede haber varias implementaciones (programas) del mismo algoritmo.

Separar el *qué* hace (la *especificación*) del *cómo* lo hace (la *implementación*) es una de las tareas más importantes del buen programador.

Esta separación es útil tanto si se trata de especificar grandes programas o sistemas como si se trata de pequeñas piezas de software que puedan usarse como partes de un programa más grande.

Llamaremos **usuario** de un software al entorno externo del software especificado, es decir, a los posibles usuarios humanos o a los posibles programas que podrían utilizar los servicios del software especificado y que, en principio, están interesados en saber *qué* hace el programa, pero no *cómo* lo hace.

La especificación de un software tiene un doble destinatario:

- Los *usuarios* del software. En este sentido, debe recoger todo lo necesario para poder usarlo conrrectamente.
- El *implementador* del software. En este sentido, describe los requisitos que cualquier implementación válida debe satisfacer; es decir, las obligaciones del implementador. Ha de dejar suficiente libertad para que éste pueda escoger la implementación que estime más adecuada con los recursos disponibles. Por eso, la especificación no debería entrar en detalles de *cómo* se debe implementar el software, ya que entonces el implementador tendría menos libertad crear su implementación.

La especificación actúa, por tanto, como una *barrera* y como un *contrato* entre los usuarios y el implementador.

3.8. Traducción y ejecución

Una vez escrito el programa, se procede a su ejecución. Para ello:

- Si el lenguaje es **compilado**: se compila, se genera el código objeto y se ejecuta éste.
- Si el lenguaje es **interpretado**: se ejecuta el código fuente directamente por medio del intérprete del lenguaje.

Si durante la compilación (o ejecución, en el caso de un lenguaje interpretado) el traductor muestra **errores en el programa fuente**, hay que volver a editar el programa, corregir los errores e intentar de nuevo.

Los errores que un traductor puede detectar son, principalmente:

- Errores **sintácticos** (por ejemplo, falta o sobra un paréntesis).

- Errores **de semántica estática** (por ejemplo, se intenta sumar una cadena a un número, detectable mediante un **chequeo de tipos**).

Ejercicio

8. Desde el punto de vista de la detección de errores sintácticos o de semántica estática, ¿qué resulta más interesante: un compilador o un intérprete? Razona la respuesta.

3.9. Pruebas

Para determinar que el programa funciona correctamente, se determina una **batería de pruebas** que debe superar el mismo para concluir que se comporta como debe.

Esas baterías de prueba (o **casos de prueba**) consisten en una serie de **datos de entrada** con los que se estimula al programa, emparejados a una serie de **resultados esperables** que se comparan con los resultados reales que el programa genera a partir de los datos de entrada.

Si los resultados obtenidos coinciden con los esperables, se concluye que el programa está funcionando **correctamente**.

En caso contrario, decimos que el programa **falla** y debemos localizar el error (o errores) que provocan el mal funcionamiento.

Las pruebas pueden detectar la presencia de errores, pero nunca pueden garantizar la ausencia de los mismos.

La verificación formal es la única forma de garantizar la ausencia de errores en un programa.

Entonces, ¿por qué hacemos pruebas?

- Para comprobar que no se han *colado* errores al codificar el algoritmo (aunque hayamos verificado la corrección del algoritmo, se nos puede haber colado un error al codificarlo en el programa).
- A veces, simplemente, no verificamos, y lo único que tenemos son las pruebas.
- También es importante comprobar la **eficiencia** del programa con ejecuciones *reales*.

3.10. Depuración

La **depuración** es el proceso de **encontrar** los errores del programa y **corregir** o eliminar dichos errores.

En caso de ser **errores sintácticos o de semántica estática**, el traductor facilita mucho la tarea de localizar la posición concreta del mismo en el programa fuente, gracias a los **mensajes de error** que genera durante la compilación o interpretación del programa.

Si tenemos un **error lógico** (un error en la lógica del programa que provoca que éste produzca resultados incorrectos), normalmente resulta más difícil de localizar.

A esos errores lógicos también se les denomina ***bugs*** («bichos», en inglés). Por eso, el proceso de depuración se denomina ***debug*** o *debugging* en inglés.

Una técnica de depuración es el **método del patito de goma**.

3.11. Documentación

La documentación es el proceso por el cual incorporamos al código fuente del programa de toda la información que pueda ayudar en la comprensión y el mantenimiento del mismo.

La documentación de un programa puede ser interna o externa:

- La **documentación interna** forma parte del código fuente del programa y se refiere al uso de **comentarios**, identificadores descriptivos, indentación, **reglas de estilo**, etc. Todo orientado a ayudar a entender el código cuando lo lea un humano.
- La **documentación externa** va fuera del código fuente e incluye análisis, diagramas de flujo y/o pseudocódigos, manuales de usuario con instrucciones para ejecutar el programa y para interpretar los resultados.

3.12. Mantenimiento

La vida útil de un programa rara vez termina cuando se ha acabado de programar y los usuarios comienzan a usarlo, sino que a partir de ese momento comienza una de las etapas más importantes y probablemente más costosas en tiempo y esfuerzo: el **mantenimiento**.

Mantener un programa consiste en realizar acciones encaminadas a:

- Corregir los fallos que puedan aparecer como consecuencia del uso del programa (fallos que no se localizaron previamente en la fase de pruebas).
- Adaptar el programa a nuevas condiciones de utilización (nuevos sistemas operativos, nuevas plataformas *hardware*...)
- Mejorar el programa incorporando nuevas funcionalidades.

Para ello, es fundamental que el programa esté correctamente documentado.

3.13. Ingeniería del software

En los años 60, los problemas a resolver son cada vez más complejos y los ordenadores son cada vez más potentes pero los programadores son incapaces de escribir programas que aprovechen esa potencia y que sean fiables.

Además resulta difícil estimar el presupuesto y el tiempo necesarios para desarrollar programas.

En 1968, Fiedrich Bauer habla por primera vez de la **crisis del software** en la primera Conferencia sobre Ingeniería del Software de la OTAN en Garmish (Alemania).

Se llega a la conclusión de que no basta con tener mejores herramientas (lenguajes), sino que hay que dar un enfoque más industrial y sistemático al desarrollo de software.

Aparece la **ingeniería del software** como disciplina.

La ingeniería del software no considera a la programación como una disciplina científica o como un arte, sino como un proceso sistemático que va más allá de escribir código.

Cuando el programa a desarrollar es grande, escribir código es sólo una de las tareas que hay que realizar. También hay que:

- Realizar un análisis del sistema.

- Estimar y planificar el tiempo y el esfuerzo necesarios para desarrollar la solución.
- Aplicar procedimientos estandarizados.
- Elaborar documentación.
- Medir la calidad del producto resultante.

El desarrollo de software complejo requiere pasar por varias etapas que, juntas, forman lo que se llama el **ciclo de vida** del software.

Respuestas a las preguntas

Respuestas a las preguntas

Bibliografía

Aguilar, Luis Joyanes. 2008. *Fundamentos de Programación*. McGraw-Hill Interamericana de España.

Pareja Flores, Cristóbal, Manuel Ojeda Aciego, Ángel Andeyro Quesada, and Carlos Rossi Jiménez. 1997. *Desarrollo de Algoritmos y Técnicas de Programación En Pascal*. Ra-Ma.

Van-Roy, Peter, and Seif Haridi. 2004. *Concepts, Techniques, and Models of Computer Programming*. MIT Press.